Catching phishing from the link alone

...and why “99% accuracy” in URL phishing detection is mostly an illusion

Data Science & Cyber · the URL / link detector (Student B) · code: github.com/segals/phishing-url-detector

Phishing is the number-one way attackers get their first foothold, and a lot of it comes down to a link. So: can you catch phishing from the **URL alone**? I built a fast, interpretable URL detector, got a near-perfect score — and then spent the rest of the project proving that score is a mirage.

> **The one-line story: F1 ≈ 1.0 in the lab, driven by a single leaked feature and by HTTP-vs-HTTPS; evaded to 0% recall by dressing a phishing URL up as a clean home-page; and on an independent dataset it collapses to F1 0.666 / AUC 0.434 — worse than a coin flip.**

1. The data, and a first red flag

I use **PhiUSIIL** (Prasad & Chandra, 2024; UCI id 967): 235,795 real URLs, 43% phishing, each with the raw URL. It is one of the largest, most recent public URL datasets. The exploratory plots already look suspicious — the two classes separate almost perfectly on trivial structural quantities, which never happens with genuinely hard data.

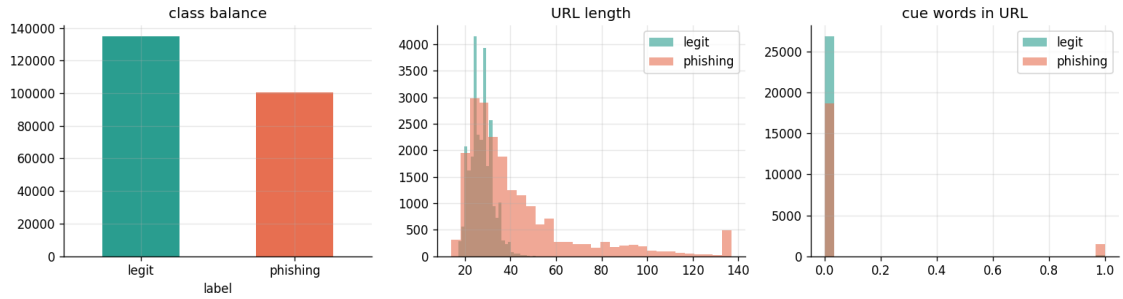

Figure 1. Class balance and two feature distributions. The classes separate far too cleanly.

2. The near-perfect score is a collection artifact

Train on the dataset’s own features and you reproduce the published ~perfect result. But take it apart: a **single** feature (`URLSimilarityIndex`) already scores F1 0.995 — it is basically the label. And even my honest, hand-built lexical features hit 0.997, because in this dataset the legitimate URLs are clean HTTPS home-pages and the phishing URLs are messy. The model learns *how the data was collected*, not what makes a link dangerous.

representation	f1	auc	f1_95ci
all 50 provided features (RF)	1.0	1.0	[1.000, 1.000]
single provided feature: URLSimilarityIndex	0.995	0.996	nan
single provided feature: URLCharProb	0.62	0.766	nan
single provided feature: TLDLegitimateProb	0.0	0.607	nan
honest raw-URL lexical (40, RF)	0.997	0.999	[0.996, 0.997]

The trivial baselines make it concrete: predicting the majority class gives 57% accuracy at **0% recall** (the accuracy paradox), and a one-line "not-HTTPS → phishing" rule already scores F1 0.68. That one artifact is doing most of the work — legitimate URLs here are 100% HTTPS, phishing only 49%.

3. What is the model actually using? SHAP says: the artifact

Because the model is a tree ensemble, I use SHAP (Lundberg & Lee, 2017) for faithful attribution. The #1 driver is `is_https` by roughly 3× the next feature, followed by path/structure. The detector is a thin veneer over the HTTP-vs-HTTPS artifact — which is exactly why the attacks below work.

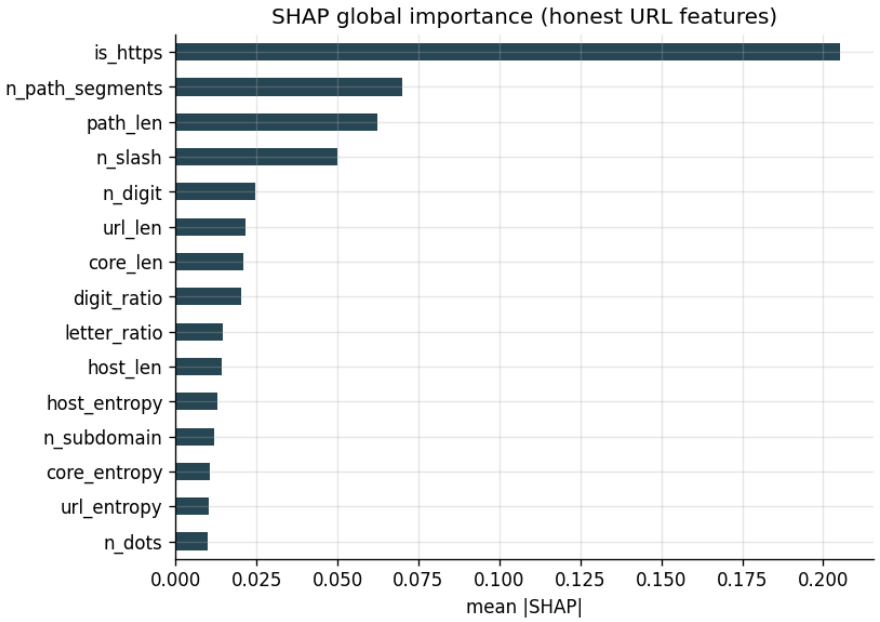

Figure 2. SHAP global importance. is_https dominates; the rest is URL structure.

4. Attack it, then defend it

A structural detector turns out to be **robust to the character tricks** that break a text model — homoglyph (Cyrillic look-alikes) and typo-squatting barely move recall (0.99). That is a genuine complementarity with the content detector, which those same tricks *do* fool. But the detector is fragile exactly where it is biased: simply serving the page over HTTPS drops recall to 0.67, and full home-page mimicry drops it to **0.00**. Normalization (the homoglyph defense) correctly does nothing here, and removing `is_https` barely helps — the bias is diffuse.

attack	recall_attacked	recall_+normalize	recall_+debias(no https)
(clean)	0.994	0.994	0.994
homoglyph	0.992	0.994	0.994
typosquat	0.993	0.993	0.994
https_upgrade	0.672	0.672	0.704
homepage_mimicry	0.0	0.0	0.0

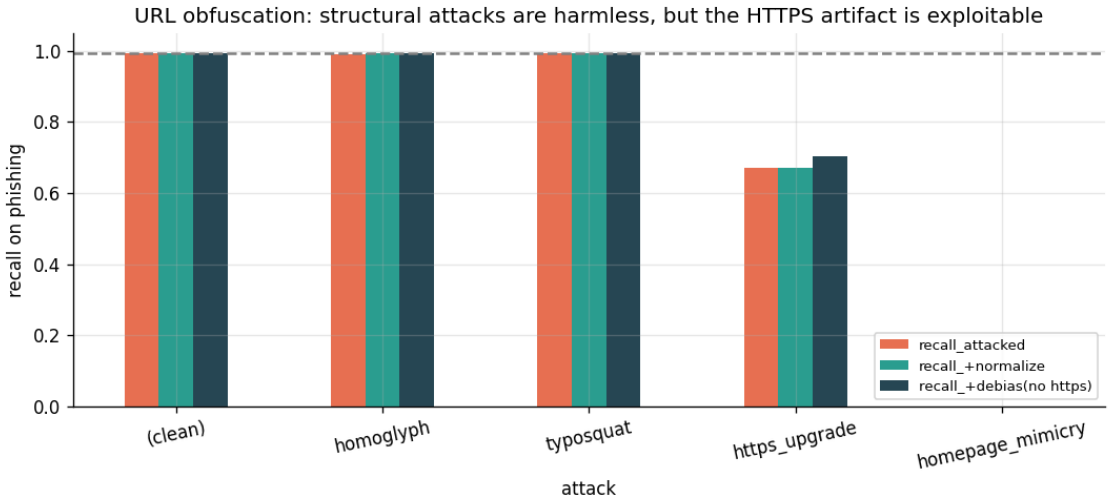

Figure 3. Structural attacks are harmless; the HTTPS artifact is the exploitable hole.

5. The reality check: it does not generalize

The real test is a **different** dataset. On the independent Kaggle Malicious-URLs corpus (Siddhartha, 2021; 651k URLs), the near-perfect model (in-distribution F1 0.997, AUC 0.999) collapses to F1 0.666 / AUC 0.434 — **worse than random**. Its ranking literally inverts, because PhiUSIIL's "home-page = legit, deep-link = phishing" artifact is reversed in another dataset. Yet it still catches **100% of live OpenPhish phishing**, because today's real phishing is still structurally messy.

setting	f1	recall_phishing	auc	threshold
in-distribution (PhiUSIIL test)	0.997	0.994	0.999	0.5
cross-dataset (Kaggle, full model)	0.666	0.999	0.434	0.5
cross-dataset (Kaggle, no is_https)	0.666	0.998	0.168	0.5
cross-dataset (Kaggle, re-thresholded)	0.667	1.0	0.434	0.01
live OpenPhish (recall only)	nan	1.0	nan	0.5

Why does it invert? Distance metrics (the course's KS and Wasserstein) make it concrete: the features that shift most between the two datasets are precisely the ones the model leans on — `is_https`, `path_len`, `n_subdomain`. The detector built its decision on the least transferable signals.

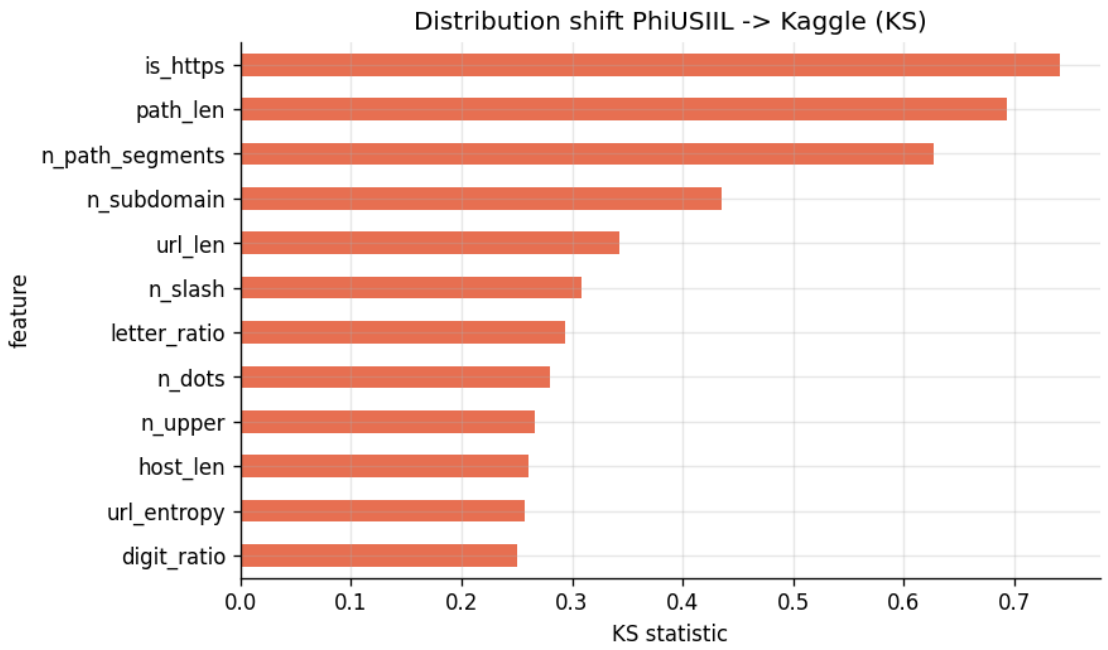

Figure 4. KS distance PhiUSIIL→Kaggle. The top-shifted features are the top SHAP drivers.

6. So what is a URL detector good for?

Honestly: as **one layer**, not the answer. It is fast (~milliseconds), private (nothing leaves the device), interpretable, robust to character-level obfuscation, and it catches today's messy phishing. But URL *structure alone is not a generalizable phishing signal* — it is trivially evaded by an attacker who hosts on legitimate HTTPS infrastructure, and it does not survive a change of dataset. The right design is **fusion**: combine this link channel with my partner's content channel and with non-lexical signals (domain age, reputation), so that when an attacker beats one channel, another still fires.

> **Bottom line: the celebrated near-perfect URL benchmarks measure dataset construction, not phishing detection. An honest URL detector is a useful, robust *second opinion* — not a standalone solution.**

Appendix — course concepts used

EDA, distributions, skewness/kurtosis, robust statistics · Pearson vs Spearman correlation, Mann–Whitney U · feature engineering, Shannon entropy, Levenshtein edit distance · the accuracy paradox, Precision/Recall/F1/F2/MCC/AUC, cost-sensitive thresholds · bootstrap CIs, McNemar · explainability (SHAP) · adversarial ML (evasion, perturbation, homoglyphs, adversarial training) · anomaly detection (Isolation Forest, LOF) · calibration · domain shift and distance metrics (KS, Wasserstein/EMD) · feature ablation and error analysis.

Data: PhiUSIIL (Prasad & Chandra 2024), Kaggle Malicious-URLs (Siddhartha 2021), OpenPhish, Majestic Million.

Methods/refs: Ma 2009; Garera 2007; Sahingoz 2019; Le 2018 (URLNet); Boucher 2022; Lundberg & Lee 2017; Unicode TR39.